

Environment Setup and SDWA Configuration

This cell initializes the SageMaker Studio environment and defines all shared configuration used throughout the SDWA ingestion pipeline. It sets the AWS region, creates a boto3 session, defines S3 bucket paths for raw and processed data, specifies the Athena/Glue database, and maps logical SDWA table names to their corresponding raw CSV filenames.

```
In [1]: # -----  
# Environment Setup: AWS region, S3 bucket paths, and session  
# -----  
  
import time  
import awswrangler as wr  
import pandas as pd  
import boto3  
  
# Create a boto3 session bound to the correct AWS region.  
region = "us-east-1"  
session = boto3.Session(region_name=region)  
  
# S3 bucket where raw CSVs and Parquet outputs live.  
bucket = "aai540-group5-public-866792937762-us-east-1-an"  
  
# S3 prefixes for raw CSVs and processed Parquet datasets.  
raw_prefix = "sdwa/raw/"  
parquet_prefix = "sdwa/parquet/"  
  
# Glue/Athena database name used for table registration.  
database = "sdwa_db"  
  
# Logical table → raw CSV filename mapping.  
sdwa_files = {  
    "events_milestones": "SDWA_EVENTS_MILESTONES.csv",  
    "violations": "SDWA_VIOLATIONS_ENFORCEMENT.csv",  
    "pws": "SDWA_PUB_WATER_SYSTEMS.csv",  
    "lcr": "SDWA_LCR_SAMPLES.csv",  
    "facilities": "SDWA_FACILITIES.csv",  
    "service_areas": "SDWA_SERVICE_AREAS.csv",  
    "geographic_areas": "SDWA_GEOGRAPHIC_AREAS.csv",  
    "ref_codes": "SDWA_REF_CODE_VALUES.csv",  
    "ansi_areas": "SDWA_REF_ANSI_AREAS.csv",  
    "pn_assoc": "SDWA_PN_VIOLATION_ASSOC.csv",  
    "site_visits": "SDWA_SITE_VISITS.csv",  
}
```

1. Utility: Determine S3 Object Size

This helper function queries S3 to retrieve the size (in MB) of a raw CSV file. The ingestion pipeline uses this size to decide whether to load the file normally or stream it in chunks,

which is important for memory-constrained environments like SageMaker Studio.

```
In [2]: # -----  
# Utility: Return S3 object size in MB  
# -----  
  
def get_s3_size_mb(key: str) -> float:  
    """  
    Query S3 metadata to determine the size of a raw CSV file.  
    Used to decide between normal vs. streaming ingestion.  
    """  
    s3 = session.client("s3")  
    resp = s3.head_object(Bucket=bucket, Key=f"{raw_prefix}{key}")  
    return resp["ContentLength"] / (1024 * 1024)
```

Helper: Read CSV with All Columns as Strings

This function reads a CSV file from S3 and coerces every column to string. SDWA datasets contain inconsistent typing, so forcing all columns to string ensures schema stability and prevents mixed-type inference issues.

```
In [3]: # -----  
# Helper: Read CSV from S3 with all columns forced to STRING  
# -----  
  
def _read_csv_all_string(key: str, nrows: int | None = None) -> pd.DataFrame:  
    """  
    Load a CSV from S3 and coerce every column to string.  
    This avoids mixed-type inference issues common in SDWA data.  
    """  
    path = f"s3://{bucket}/{raw_prefix}{key}"  
  
    df = wr.s3.read_csv(  
        path,  
        dtype=str,          # Force all columns to string  
        low_memory=False,   # Avoid partial type inference  
        nrows=nrows  
    )  
  
    # Ensure dtype consistency across chunks  
    df = df.astype(str)  
    return df  
  
# -----  
# Helper: Write DataFrame to Parquet (all STRING columns)  
# -----  
  
def _write_parquet_all_string(table_name: str, df: pd.DataFrame) -> None:  
    """  
    Write a DataFrame to S3 as a Parquet dataset.  
    All columns remain STRING to ensure schema stability in Athena.  
    """
```

```

out_path = f"s3://{bucket}/{parquet_prefix}{table_name}/"
print(f" - Writing Parquet to: {out_path}")

wr.s3.to_parquet(
    df=df,
    path=out_path,
    dataset=True,
    mode="overwrite"
)

print(f" - Parquet write complete for '{table_name}'.")

```

Normal Ingestion for Small and Medium SDWA CSV Files

This cell defines the ingestion path used for SDWA tables that comfortably fit into memory. It reads the entire CSV from S3 in one operation, forces all columns to STRING to maintain schema consistency across the pipeline, logs the row count for traceability, and writes the resulting DataFrame to S3 as a Parquet dataset. This is the simplest and fastest ingestion mode and is used whenever the file size falls below the streaming threshold.

```

In [4]: # -----
# Normal ingestion path for smaller CSVs
# -----

def ingest_normal_csv(table_name: str, filename: str) -> None:
    """
    Load a CSV fully into memory, convert to STRING columns,
    and write to Parquet.
    """
    path = f"s3://{bucket}/{raw_prefix}{filename}"
    print(f" - Reading CSV (normal): {path}")

    df = _read_csv_all_string(filename)
    print(f" - Loaded {len(df):,} rows for '{table_name}'.")

    _write_parquet_all_string(table_name, df)

```

Streaming Ingestion for Large SDWA CSV Files

This cell defines the ingestion path used for SDWA tables that are too large to fit into memory (multi-hundred-MB to multi-GB files). Instead of loading the entire CSV at once, the file is streamed from S3 in fixed-size chunks. Each chunk is converted to STRING columns for schema consistency and appended to a Parquet dataset in S3. This approach prevents memory exhaustion in SageMaker Studio while still producing a single unified Parquet table for downstream Athena queries and ML workflows.

```

In [5]: # -----
# Streaming ingestion for large CSVs (chunked processing)

```

```
# -----
def ingest_large_csv_streaming(table_name: str, filename: str, chunksize: int = 200
    """
    Stream a large CSV from S3 in chunks to avoid memory pressure.
    Each chunk is coerced to STRING and appended to a Parquet dataset.
    """

    path = f"s3://{bucket}/{raw_prefix}{filename}"
    print(f" - Streaming CSV from: {path}")

    chunk_iter = wr.s3.read_csv(
        path,
        dtype=str,          # Force all columns to string
        low_memory=False,
        chunksize=chunksize
    )

    out_path = f"s3://{bucket}/{parquet_prefix}{table_name}/"
    print(f" - Writing Parquet chunks to: {out_path}")

    first = True
    total_rows = 0
    chunk_idx = 0

    for chunk in chunk_iter:
        chunk_idx += 1
        chunk = chunk.astype(str) # Ensure dtype consistency
        rows = len(chunk)
        total_rows += rows

        mode = "overwrite" if first else "append"
        first = False

        wr.s3.to_parquet(
            df=chunk,
            path=out_path,
            dataset=True,
            mode=mode
        )

        print(f"[chunk {chunk_idx}] wrote {rows:,} rows (mode={mode}).")

    print(f" - Completed streaming ingest for '{table_name}' ({total_rows:,} total
```

Ingestion Router: Select Normal or Streaming Mode and Execute Full Pipeline

This cell defines the routing logic that determines whether each SDWA table should be ingested using the normal in-memory method or the streaming chunk-based method. The decision is based on the file's size in S3 relative to a configurable threshold (default 500 MB). After defining the router, the cell iterates over all SDWA tables listed in `sdwa_files` and

ingests each one accordingly. This provides a single, automated entry point for the entire ingestion workflow.

```
In [6]: # -----
# Route ingestion based on file size
# -----

def ingest_table(table_name: str, filename: str, threshold_mb: float = 500.0):
    """
    Decide whether to ingest a table normally or via streaming.
    Threshold defaults to 500 MB.
    """
    size = get_s3_size_mb(filename)

    print(f"\n=== Ingesting {table_name} ({filename}) ===")
    print(f"File size: {size:.2f} MB")

    if size > threshold_mb:
        print(f"Using STREAMING ingestion for {table_name}...")
        ingest_large_csv_streaming(table_name, filename)
    else:
        print(f"Using NORMAL ingestion for {table_name}...")
        ingest_normal_csv(table_name, filename)

    print(f"Completed ingestion for {table_name}.")

# -----
# Run ingestion for all SDWA tables
# -----

for key, filename in sdwa_files.items():
    ingest_table(key, filename)

=== Ingesting events_milestones (SDWA_EVENTS_MILESTONES.csv) ===
File size: 65.76 MB
Using NORMAL ingestion for events_milestones...
- Reading CSV (normal): s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/ra
w/SDWA_EVENTS_MILESTONES.csv

2026-06-03 19:39:15,397 WARNING services.py:2137 -- WARNING: The object store is usi
ng /tmp instead of /dev/shm because /dev/shm has only 1908383744 bytes available. Th
is will harm performance! You may be able to free up space by deleting files in /de
v/shm. If you are inside a Docker container, you can increase /dev/shm size by passi
ng '--shm-size=4.58gb' to 'docker run' (or add it to the run_options list in a Ray c
luster config). Make sure to set this to more than 30% of available RAM.
2026-06-03 19:39:15,539 INFO worker.py:2007 -- Started a local Ray instance.
/opt/conda/lib/python3.12/site-packages/ray/_private/worker.py:2046: FutureWarning:
Tip: In future versions of Ray, Ray will no longer override accelerator visible devi
ces env var if num_gpus=0 or num_gpus=None (default). To enable this behavior and tu
rn off this error message, set RAY_ACCEL_ENV_VAR_OVERRIDE_ON_ZERO=0
warnings.warn(
```

- Loaded 360,370 rows for 'events_milestones'.
- Writing Parquet to: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/events_milestones/
- Parquet write complete for 'events_milestones'.

Completed ingestion for events_milestones.

=== Ingesting violations (SDWA_VIOLATIONS_ENFORCEMENT.csv) ===

File size: 3886.44 MB

Using STREAMING ingestion for violations...

- Streaming CSV from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_VIOLATIONS_ENFORCEMENT.csv

- Writing Parquet chunks to: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/violations/

[chunk 1] wrote 200,000 rows (mode=overwrite).

[chunk 2] wrote 200,000 rows (mode=append).

[chunk 3] wrote 200,000 rows (mode=append).

[chunk 4] wrote 200,000 rows (mode=append).

[chunk 5] wrote 200,000 rows (mode=append).

[chunk 6] wrote 200,000 rows (mode=append).

[chunk 7] wrote 200,000 rows (mode=append).

[chunk 8] wrote 200,000 rows (mode=append).

[chunk 9] wrote 200,000 rows (mode=append).

[chunk 10] wrote 200,000 rows (mode=append).

[chunk 11] wrote 200,000 rows (mode=append).

[chunk 12] wrote 200,000 rows (mode=append).

[chunk 13] wrote 200,000 rows (mode=append).

[chunk 14] wrote 200,000 rows (mode=append).

[chunk 15] wrote 200,000 rows (mode=append).

[chunk 16] wrote 200,000 rows (mode=append).

[chunk 17] wrote 200,000 rows (mode=append).

[chunk 18] wrote 200,000 rows (mode=append).

[chunk 19] wrote 200,000 rows (mode=append).

[chunk 20] wrote 200,000 rows (mode=append).

[chunk 21] wrote 200,000 rows (mode=append).

[chunk 22] wrote 200,000 rows (mode=append).

[chunk 23] wrote 200,000 rows (mode=append).

[chunk 24] wrote 200,000 rows (mode=append).

[chunk 25] wrote 200,000 rows (mode=append).

[chunk 26] wrote 200,000 rows (mode=append).

[chunk 27] wrote 200,000 rows (mode=append).

[chunk 28] wrote 200,000 rows (mode=append).

[chunk 29] wrote 200,000 rows (mode=append).

[chunk 30] wrote 200,000 rows (mode=append).

[chunk 31] wrote 200,000 rows (mode=append).

[chunk 32] wrote 200,000 rows (mode=append).

[chunk 33] wrote 200,000 rows (mode=append).

[chunk 34] wrote 200,000 rows (mode=append).

[chunk 35] wrote 200,000 rows (mode=append).

[chunk 36] wrote 200,000 rows (mode=append).

[chunk 37] wrote 200,000 rows (mode=append).

[chunk 38] wrote 200,000 rows (mode=append).

[chunk 39] wrote 200,000 rows (mode=append).

[chunk 40] wrote 200,000 rows (mode=append).

[chunk 41] wrote 200,000 rows (mode=append).

[chunk 42] wrote 200,000 rows (mode=append).

[chunk 43] wrote 200,000 rows (mode=append).

[chunk 44] wrote 200,000 rows (mode=append).
[chunk 45] wrote 200,000 rows (mode=append).
[chunk 46] wrote 200,000 rows (mode=append).
[chunk 47] wrote 200,000 rows (mode=append).
[chunk 48] wrote 200,000 rows (mode=append).
[chunk 49] wrote 200,000 rows (mode=append).
[chunk 50] wrote 200,000 rows (mode=append).
[chunk 51] wrote 200,000 rows (mode=append).
[chunk 52] wrote 200,000 rows (mode=append).
[chunk 53] wrote 200,000 rows (mode=append).
[chunk 54] wrote 200,000 rows (mode=append).
[chunk 55] wrote 200,000 rows (mode=append).
[chunk 56] wrote 200,000 rows (mode=append).
[chunk 57] wrote 200,000 rows (mode=append).
[chunk 58] wrote 200,000 rows (mode=append).
[chunk 59] wrote 200,000 rows (mode=append).
[chunk 60] wrote 200,000 rows (mode=append).
[chunk 61] wrote 200,000 rows (mode=append).
[chunk 62] wrote 200,000 rows (mode=append).
[chunk 63] wrote 200,000 rows (mode=append).
[chunk 64] wrote 200,000 rows (mode=append).
[chunk 65] wrote 200,000 rows (mode=append).
[chunk 66] wrote 200,000 rows (mode=append).
[chunk 67] wrote 200,000 rows (mode=append).
[chunk 68] wrote 200,000 rows (mode=append).
[chunk 69] wrote 200,000 rows (mode=append).
[chunk 70] wrote 200,000 rows (mode=append).
[chunk 71] wrote 200,000 rows (mode=append).
[chunk 72] wrote 200,000 rows (mode=append).
[chunk 73] wrote 200,000 rows (mode=append).
[chunk 74] wrote 200,000 rows (mode=append).
[chunk 75] wrote 200,000 rows (mode=append).
[chunk 76] wrote 200,000 rows (mode=append).
[chunk 77] wrote 98,031 rows (mode=append).
- Completed streaming ingest for 'violations' (15,298,031 total rows).
Completed ingestion for violations.

=== Ingesting pws (SDWA_PUB_WATER_SYSTEMS.csv) ===
File size: 123.12 MB
Using NORMAL ingestion for pws...
- Reading CSV (normal): s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_PUB_WATER_SYSTEMS.csv
- Loaded 433,698 rows for 'pws'.
- Writing Parquet to: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/pws/
- Parquet write complete for 'pws'.
Completed ingestion for pws.

=== Ingesting lcr (SDWA_LCR_SAMPLES.csv) ===
File size: 118.26 MB
Using NORMAL ingestion for lcr...
- Reading CSV (normal): s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_LCR_SAMPLES.csv
- Loaded 924,498 rows for 'lcr'.
- Writing Parquet to: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/lcr/

- Parquet write complete for 'lcr'.
Completed ingestion for lcr.

=== Ingesting facilities (SDWA_FACILITIES.csv) ===

File size: 159.23 MB

Using NORMAL ingestion for facilities...

- Reading CSV (normal): s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_FACILITIES.csv

- Loaded 1,550,159 rows for 'facilities'.

- Writing Parquet to: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/facilities/

- Parquet write complete for 'facilities'.

Completed ingestion for facilities.

=== Ingesting service_areas (SDWA_SERVICE_AREAS.csv) ===

File size: 18.42 MB

Using NORMAL ingestion for service_areas...

- Reading CSV (normal): s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_SERVICE_AREAS.csv

- Loaded 422,099 rows for 'service_areas'.

- Writing Parquet to: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/service_areas/

- Parquet write complete for 'service_areas'.

Completed ingestion for service_areas.

=== Ingesting geographic_areas (SDWA_GEOGRAPHIC_AREAS.csv) ===

File size: 38.84 MB

Using NORMAL ingestion for geographic_areas...

- Reading CSV (normal): s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_GEOGRAPHIC_AREAS.csv

- Loaded 577,661 rows for 'geographic_areas'.

- Writing Parquet to: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/geographic_areas/

- Parquet write complete for 'geographic_areas'.

Completed ingestion for geographic_areas.

=== Ingesting ref_codes (SDWA_REF_CODE_VALUES.csv) ===

File size: 0.12 MB

Using NORMAL ingestion for ref_codes...

- Reading CSV (normal): s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_REF_CODE_VALUES.csv

- Loaded 2,376 rows for 'ref_codes'.

- Writing Parquet to: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/ref_codes/

- Parquet write complete for 'ref_codes'.

Completed ingestion for ref_codes.

=== Ingesting ansi_areas (SDWA_REF_ANSI_AREAS.csv) ===

File size: 0.08 MB

Using NORMAL ingestion for ansi_areas...

- Reading CSV (normal): s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_REF_ANSI_AREAS.csv

- Loaded 3,235 rows for 'ansi_areas'.

- Writing Parquet to: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/ansi_areas/

- Parquet write complete for 'ansi_areas'.

Completed ingestion for ansi_areas.

=== Ingesting pn_assoc (SDWA_PN_VIOLATION_ASSOC.csv) ===

File size: 45.99 MB

Using NORMAL ingestion for pn_assoc...

- Reading CSV (normal): s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_PN_VIOLATION_ASSOC.csv

- Loaded 378,063 rows for 'pn_assoc'.

- Writing Parquet to: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/pn_assoc/

- Parquet write complete for 'pn_assoc'.

Completed ingestion for pn_assoc.

=== Ingesting site_visits (SDWA_SITE_VISITS.csv) ===

File size: 371.89 MB

Using NORMAL ingestion for site_visits...

- Reading CSV (normal): s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_SITE_VISITS.csv

- Loaded 2,478,266 rows for 'site_visits'.

- Writing Parquet to: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/site_visits/

- Parquet write complete for 'site_visits'.

Completed ingestion for site_visits.

Register Parquet Datasets as Athena Tables (All Columns as STRING)

This cell defines a helper function that registers a Parquet dataset as an Athena table in the AWS Glue Catalog. It selects a sample Parquet file to infer the column names, maps every column to the STRING type to maintain schema consistency across all SDWA tables, and then creates or overwrites the Athena table. This ensures that downstream SQL queries operate on a stable, predictable schema regardless of the original CSV typing.

```
In [7]: # -----  
# Register Parquet datasets as Athena tables (all STRING schema)  
# -----  
  
def safe_create_parquet_table(database: str, table: str, path: str) -> None:  
    """  
    Register a Parquet dataset as an Athena table.  
    All columns are mapped to STRING to ensure schema stability.  
    """  
  
    start = time.time()  
    print(f"\n[START] Registering Athena table '{table}'")  
    print(f"S3 Path: {path}")  
  
    objects = wr.s3.list_objects(path)  
    if not objects:  
        print(f"[ERROR] No Parquet files found under {path}")  
        return  
  
    sample_path = objects[0]  
    print(f" - Using sample file: {sample_path}")
```

```

sample = wr.s3.read_parquet(sample_path)

# Map all columns to Athena STRING type
columns_types = {col: "string" for col in sample.columns}
print(f" - Columns detected: {len(columns_types)} (all coerced to STRING)")

wr.catalog.create_parquet_table(
    database=database,
    table=table,
    path=path,
    columns_types=columns_types,
    mode="overwrite"
)

elapsed = time.time() - start
print(f"[DONE] Table '{table}' registered successfully in {elapsed:.2f}s")

```

Ensure Athena/Glue Database Exists (Idempotent Check)

This cell verifies that the configured Athena/Glue database is present in the AWS Glue Catalog. If the database does not already exist, it is created.

If it *does* exist, the cell simply reports that fact.

Because this check is idempotent, it can be safely re-run without affecting any existing tables or metadata.

```

In [8]: # -----
# Ensure Glue/Athena database exists (idempotent)
# -----
dbs = wr.catalog.databases()

if database not in dbs["Database"].values:
    print(f"Creating database '{database}' in Glue Catalog...")
    wr.catalog.create_database(name=database)
else:
    print(f"Database '{database}' already exists.")

```

Database 'sdwa_db' already exists.

Athena Schema Sanity Check for All Registered SDWA Tables

This cell performs a lightweight validation step to ensure that each Parquet dataset was ingested and registered correctly. For every SDWA table, it reads a small sample (one chunk) from the Parquet dataset and prints the pandas dtypes. Because the ingestion pipeline forces all columns to STRING, every column should appear as `object` / `string` here. This confirms schema consistency across tables and prevents downstream Athena query failures caused by mixed or unexpected types.

```
In [9]: # -----  
# Athena Sanity Check: Confirm Parquet schemas match expectations  
# -----  
# This loop reads a small sample from each Parquet dataset to verify:  
#   1. Columns exist as expected  
#   2. ALL columns are STRING (as enforced during ingestion)  
#   3. Athena/Glue will interpret the schema consistently  
#  
# This is critical in SageMaker Studio because schema mismatches  
# can cause Athena query failures later in the ML pipeline.  
  
for table_name in sdwa_files.keys():  
    path = f"s3://{bucket}/{parquet_prefix}{table_name}/"  
    print(f"\n=== Schema check for '{table_name}' ===")  
  
    # Read only the first chunk to avoid loading large datasets into memory  
    df_iter = wr.s3.read_parquet(path, dataset=True, chunked=True)  
    df_sample = next(df_iter)  
  
    # Display column names and dtypes (should all be object/string)  
    print(df_sample.dtypes)
```

```

=== Schema check for 'events_milestones' ===
SUBMISSIONYEARQUARTER    string[python]
PWSID                    string[python]
EVENT_SCHEDULE_ID        string[python]
EVENT_END_DATE            string[python]
EVENT_ACTUAL_DATE        string[python]
EVENT_COMMENTS_TEXT      string[python]
EVENT_MILESTONE_CODE      string[python]
EVENT_REASON_CODE        string[python]
FIRST_REPORTED_DATE      string[python]
LAST_REPORTED_DATE       string[python]
dtype: object

```

```

=== Schema check for 'violations' ===
SUBMISSIONYEARQUARTER    string[python]
PWSID                    string[python]
VIOLATION_ID            string[python]
FACILITY_ID             string[python]
COMPL_PER_BEGIN_DATE     string[python]
COMPL_PER_END_DATE       string[python]
NON_COMPL_PER_BEGIN_DATE string[python]
NON_COMPL_PER_END_DATE   string[python]
PWS_DEACTIVATION_DATE    string[python]
VIOLATION_CODE           string[python]
VIOLATION_CATEGORY_CODE  string[python]
IS_HEALTH_BASED_IND      string[python]
CONTAMINANT_CODE         string[python]
VIOL_MEASURE             string[python]
UNIT_OF_MEASURE          string[python]
FEDERAL_MCL             string[python]
STATE_MCL               string[python]
IS_MAJOR_VIOL_IND        string[python]
SEVERITY_IND_CNT         string[python]
CALCULATED_RTC_DATE      string[python]
VIOLATION_STATUS         string[python]
PUBLIC_NOTIFICATION_TIER string[python]
CALCULATED_PUB_NOTIF_TIER string[python]
VIOL_ORIGINATOR_CODE     string[python]
SAMPLE_RESULT_ID         string[python]
CORRECTIVE_ACTION_ID     string[python]
RULE_CODE               string[python]
RULE_GROUP_CODE          string[python]
RULE_FAMILY_CODE         string[python]
VIOL_FIRST_REPORTED_DATE string[python]
VIOL_LAST_REPORTED_DATE  string[python]
ENFORCEMENT_ID          string[python]
ENFORCEMENT_DATE         string[python]
ENFORCEMENT_ACTION_TYPE_CODE string[python]
ENF_ACTION_CATEGORY      string[python]
ENF_ORIGINATOR_CODE      string[python]
ENF_FIRST_REPORTED_DATE  string[python]
ENF_LAST_REPORTED_DATE   string[python]
dtype: object

```

```

=== Schema check for 'pws' ===
SUBMISSIONYEARQUARTER    string[python]

```

PWSID	string[python]
PWS_NAME	string[python]
PRIMACY_AGENCY_CODE	string[python]
EPA_REGION	string[python]
SEASON_BEGIN_DATE	string[python]
SEASON_END_DATE	string[python]
PWS_ACTIVITY_CODE	string[python]
PWS_DEACTIVATION_DATE	string[python]
PWS_TYPE_CODE	string[python]
DBPR_SCHEDULE_CAT_CODE	string[python]
CDS_ID	string[python]
GW_SW_CODE	string[python]
LT2_SCHEDULE_CAT_CODE	string[python]
OWNER_TYPE_CODE	string[python]
POPULATION_SERVED_COUNT	string[python]
POP_CAT_2_CODE	string[python]
POP_CAT_3_CODE	string[python]
POP_CAT_4_CODE	string[python]
POP_CAT_5_CODE	string[python]
POP_CAT_11_CODE	string[python]
PRIMACY_TYPE	string[python]
PRIMARY_SOURCE_CODE	string[python]
IS_GRANT_ELIGIBLE_IND	string[python]
IS_WHOLESALE_IND	string[python]
IS_SCHOOL_OR_DAYCARE_IND	string[python]
SERVICE_CONNECTIONS_COUNT	string[python]
SUBMISSION_STATUS_CODE	string[python]
ORG_NAME	string[python]
ADMIN_NAME	string[python]
EMAIL_ADDR	string[python]
PHONE_NUMBER	string[python]
PHONE_EXT_NUMBER	string[python]
FAX_NUMBER	string[python]
ALT_PHONE_NUMBER	string[python]
ADDRESS_LINE1	string[python]
ADDRESS_LINE2	string[python]
CITY_NAME	string[python]
ZIP_CODE	string[python]
COUNTRY_CODE	string[python]
FIRST_REPORTED_DATE	string[python]
LAST_REPORTED_DATE	string[python]
STATE_CODE	string[python]
SOURCE_WATER_PROTECTION_CODE	string[python]
SOURCE_PROTECTION_BEGIN_DATE	string[python]
OUTSTANDING_PERFORMER	string[python]
OUTSTANDING_PERFORM_BEGIN_DATE	string[python]
REDUCED_RTCT_MONITORING	string[python]
REDUCED_MONITORING_BEGIN_DATE	string[python]
REDUCED_MONITORING_END_DATE	string[python]
SEASONAL_STARTUP_SYSTEM	string[python]
dtype: object	

=== Schema check for 'lcr' ===

SUBMISSIONYEARQUARTER	string[python]
PWSID	string[python]
SAMPLE_ID	string[python]

SAMPLING_END_DATE	string[python]
SAMPLING_START_DATE	string[python]
RECONCILIATION_ID	string[python]
SAMPLE_FIRST_REPORTED_DATE	string[python]
SAMPLE_LAST_REPORTED_DATE	string[python]
SAR_ID	string[python]
CONTAMINANT_CODE	string[python]
RESULT_SIGN_CODE	string[python]
SAMPLE_MEASURE	string[python]
UNIT_OF_MEASURE	string[python]
SAR_FIRST_REPORTED_DATE	string[python]
SAR_LAST_REPORTED_DATE	string[python]

dtype: object

=== Schema check for 'facilities' ===

SUBMISSIONYEARQUARTER	string[python]
PWSID	string[python]
FACILITY_ID	string[python]
FACILITY_NAME	string[python]
STATE_FACILITY_ID	string[python]
FACILITY_ACTIVITY_CODE	string[python]
FACILITY_DEACTIVATION_DATE	string[python]
FACILITY_TYPE_CODE	string[python]
SUBMISSION_STATUS_CODE	string[python]
IS_SOURCE_IND	string[python]
WATER_TYPE_CODE	string[python]
AVAILABILITY_CODE	string[python]
SELLER_TREATMENT_CODE	string[python]
SELLER_PWSID	string[python]
SELLER_PWS_NAME	string[python]
FILTRATION_STATUS_CODE	string[python]
IS_SOURCE_TREATED_IND	string[python]
FIRST_REPORTED_DATE	string[python]
LAST_REPORTED_DATE	string[python]

dtype: object

=== Schema check for 'service_areas' ===

SUBMISSIONYEARQUARTER	string[python]
PWSID	string[python]
SERVICE_AREA_TYPE_CODE	string[python]
IS_PRIMARY_SERVICE_AREA_CODE	string[python]
FIRST_REPORTED_DATE	string[python]
LAST_REPORTED_DATE	string[python]

dtype: object

=== Schema check for 'geographic_areas' ===

SUBMISSIONYEARQUARTER	string[python]
PWSID	string[python]
GEO_ID	string[python]
AREA_TYPE_CODE	string[python]
TRIBAL_CODE	string[python]
STATE_SERVED	string[python]
ANSI_ENTITY_CODE	string[python]
ZIP_CODE_SERVED	string[python]
CITY_SERVED	string[python]
COUNTY_SERVED	string[python]

```
LAST_REPORTED_DATE      string[python]
dtype: object
```

```
=== Schema check for 'ref_codes' ===
VALUE_TYPE              string[python]
VALUE_CODE              string[python]
VALUE_DESCRIPTION       string[python]
dtype: object
```

```
=== Schema check for 'ansi_areas' ===
ANSI_STATE_CODE         string[python]
ANSI_ENTITY_CODE        string[python]
ANSI_NAME               string[python]
STATE_CODE              string[python]
dtype: object
```

```
=== Schema check for 'pn_assoc' ===
SUBMISSIONYEARQUARTER   string[python]
PWSID                   string[python]
PN_VIOLATION_ID         string[python]
RELATED_VIOLATION_ID    string[python]
COMPL_PER_BEGIN_DATE    string[python]
COMPL_PER_END_DATE      string[python]
NON_COMPL_PER_BEGIN_DATE string[python]
NON_COMPL_PER_END_DATE  string[python]
VIOLATION_CODE          string[python]
CONTAMINANT_CODE        string[python]
FIRST_REPORTED_DATE     string[python]
LAST_REPORTED_DATE      string[python]
dtype: object
```

```
=== Schema check for 'site_visits' ===
SUBMISSIONYEARQUARTER   string[python]
PWSID                   string[python]
VISIT_ID                string[python]
VISIT_DATE              string[python]
AGENCY_TYPE_CODE        string[python]
VISIT_REASON_CODE       string[python]
MANAGEMENT_OPS_EVAL_CODE string[python]
SOURCE_WATER_EVAL_CODE  string[python]
SECURITY_EVAL_CODE      string[python]
PUMPS_EVAL_CODE         string[python]
OTHER_EVAL_CODE         string[python]
COMPLIANCE_EVAL_CODE    string[python]
DATA_VERIFICATION_EVAL_CODE string[python]
TREATMENT_EVAL_CODE     string[python]
FINISHED_WATER_STOR_EVAL_CODE string[python]
DISTRIBUTION_EVAL_CODE  string[python]
FINANCIAL_EVAL_CODE     string[python]
VISIT_COMMENTS          string[python]
FIRST_REPORTED_DATE     string[python]
LAST_REPORTED_DATE      string[python]
dtype: object
```

Athena Workgroup and Output Configuration

This cell defines the Athena execution environment used for all SQL queries in the notebook. It sets the workgroup (defaulting to "primary") and specifies the S3 location where Athena query results will be written. Printing these values provides a quick sanity check to ensure queries will execute in the expected context and store results in the correct output bucket.

```
In [10]: # -----  
# Athena configuration  
# -----  
athena_workgroup = "primary"  
athena_output = f"s3://{bucket}/athena/results/"  
  
print(f"Athena workgroup: {athena_workgroup}")  
print(f"Athena output: {athena_output}")
```

```
Athena workgroup: primary  
Athena output: s3://aai540-group5-public-866792937762-us-east-1-an/athena/results/
```

List All Tables Registered in the Athena Database

This cell performs a quick verification step by querying the AWS Glue Catalog for all tables associated with the configured Athena database. Displaying the database/table pairs confirms that each SDWA dataset was successfully registered during the ingestion and catalog-creation steps, and provides a convenient reference for subsequent Athena queries.

```
In [11]: # -----  
# Athena test: list tables in the database  
# -----  
tables = wr.catalog.tables(database=database)  
print(tables[["Database", "Table"]])
```

	Database	Table
0	sdwa_db	ansi_areas
1	sdwa_db	events_milestones
2	sdwa_db	facilities
3	sdwa_db	geographic_areas
4	sdwa_db	lcr
5	sdwa_db	pn_assoc
6	sdwa_db	pws
7	sdwa_db	pws_master_prod
8	sdwa_db	pws_master_test
9	sdwa_db	pws_master_train
10	sdwa_db	pws_master_val
11	sdwa_db	ref_codes
12	sdwa_db	service_areas
13	sdwa_db	site_visits
14	sdwa_db	temp_table_24b7e38abcb34ee1afa1f23d19e99373
15	sdwa_db	violations

Preview a Sample of Rows from Each Athena Table

This cell performs a quick validation step by querying Athena for the first five rows of every SDWA table registered in the Glue Catalog. It confirms that each table is readable, that the Parquet datasets were registered correctly, and that Athena can successfully execute SQL queries against them. This provides an early sanity check before performing more complex joins or feature engineering.

```
In [12]: # -----  
# Athena test: sample a few rows from each table  
# -----  
for table_name in sdwa_files.keys():  
    print(f"\n=== Athena preview: {table_name} ===")  
    query = f"SELECT * FROM {database}.{table_name} LIMIT 5"  
    df_preview = wr.athena.read_sql_query(  
        sql=query,  
        database=database,  
        workgroup=athena_workgroup,  
        s3_output=athena_output,  
    )  
    print(df_preview)
```

=== Athena preview: events_milestones ===

	submissionyearquarter	pwsid	event_schedule_id	event_end_date	\
0	2026Q1	AL0001223	TTT-AL1813870	09/07/2022	
1	2026Q1	AL0001566	TTT-AL1816160	10/24/2022	
2	2026Q1	IN2492893	TTT-IN516540	10/19/2022	
3	2026Q1	IN2520141	TTT-IN544720	04/06/2023	
4	2026Q1	IN2860031	TTT-IN598640	08/22/2024	

	event_actual_date		event_comments_text	\
0	10/06/2022	SUBMIT LEVEL 1 TCR ASSESSMENT;0;Sample Lab ID:...		
1	11/19/2022	SUBMIT LEVEL 1 TCR ASSESSMENT;0;Sample Lab ID:...		
2	11/03/2022	LVL2 TTT MULTIPLE TC+ 2ND LVL 1-12 MN;0;Sample...		
3	04/16/2023	LVL2 TTT TC+/EC- WO RPTS 2ND LVL 1-12 MN;0;Sam...		
4	09/15/2024	LVL2 TTT MULTIPLE TC+ 2ND LVL 1-12 MN;0;Sample...		

	event_milestone_code	event_reason_code	first_reported_date	\
0	RTL1	L1TD	02/23/2023	
1	RTL1	L1TD	02/23/2023	
2	RTL2	L2TB	03/07/2025	
3	RTL2	L2TB	03/07/2025	
4	RTL2	L2TB	03/07/2025	

	last_reported_date
0	11/18/2025
1	03/30/2026
2	12/17/2025
3	02/24/2026
4	02/24/2026

=== Athena preview: violations ===

	submissionyearquarter	pwsid	violation_id	facility_id	\
0	2026Q1	IL0730100	nan	nan	
1	2026Q1	IL0730100	nan	nan	
2	2026Q1	IL0730100	nan	nan	
3	2026Q1	IL0730100	nan	nan	
4	2026Q1	IL0730100	nan	nan	

	compl_per_begin_date	compl_per_end_date	non_compl_per_begin_date	\
0	nan	nan	nan	
1	nan	nan	nan	
2	nan	nan	nan	
3	nan	nan	nan	
4	nan	nan	nan	

	non_compl_per_end_date	pws_deactivation_date	violation_code	...	\
0	nan	nan	nan	...	
1	nan	nan	nan	...	
2	nan	nan	nan	...	
3	nan	nan	nan	...	
4	nan	nan	nan	...	

	rule_family_code	viol_first_reported_date	viol_last_reported_date	\
0	nan	nan	nan	
1	nan	nan	nan	
2	nan	nan	nan	
3	nan	nan	nan	

4	nan	nan	nan
---	-----	-----	-----

	enforcement_id	enforcement_date	enforcement_action_type_code	\
0	300289	09/09/1989	SIA	
1	383	09/30/1983	SIF	
2	3E81	09/30/1981	SIF	
3	300389	09/09/1989	SIE	
4	381	09/30/1981	SIF	

	enf_action_category	enf_originator_code	enf_first_reported_date	\
0	nan	S	01/03/1990	
1	nan	S	12/09/2005	
2	nan	S	09/30/1981	
3	nan	S	01/03/1990	
4	nan	S	12/09/2005	

	enf_last_reported_date
0	12/06/2006
1	12/06/2006
2	09/30/1981
3	12/06/2006
4	12/06/2006

[5 rows x 38 columns]

=== Athena preview: pws ===

	submissionyearquarter	pwsid	pws_name	\
0	2026Q1	NY5906829	VILLAGE PLAZA-LINCOLNDALE	
1	2026Q1	LA1017090	BRADY MOBILE HOME PARK	
2	2026Q1	NY3530099	BULLVILLE GROCERY	
3	2026Q1	NY5930099	265 ROUTE 202	
4	2026Q1	NY3100574	PENDLETON TOWN WD	

	primacy_agency_code	epa_region	season_begin_date	season_end_date	\
0	NY	02	01-01	12-31	
1	LA	06	nan	nan	
2	NY	02	01-01	12-31	
3	NY	02	01-01	12-31	
4	NY	02	nan	nan	

	pws_activity_code	pws_deactivation_date	pws_type_code	...	\
0	A	nan	TNCWS	...	
1	A	nan	CWS	...	
2	A	nan	TNCWS	...	
3	A	nan	TNCWS	...	
4	A	nan	CWS	...	

	last_reported_date	state_code	source_water_protection_code	\
0	03/27/2026	NY	N	
1	03/31/2026	LA	nan	
2	03/27/2026	NY	N	
3	03/27/2026	NY	N	
4	03/27/2026	NY	Y	

	source_protection_begin_date	outstanding_performer	\
0	nan	nan	

1	nan	nan
2	nan	nan
3	nan	nan
4	06/23/2009	nan

outstanding_perform_begin_date reduced_rtc_r_monitoring \		
0	nan	nan
1	nan	nan
2	nan	nan
3	nan	nan
4	nan	nan

reduced_monitoring_begin_date reduced_monitoring_end_date \		
0	nan	nan
1	nan	nan
2	nan	nan
3	nan	nan
4	nan	nan

seasonal_startup_system	
0	nan
1	nan
2	nan
3	nan
4	nan

[5 rows x 51 columns]

=== Athena preview: lcr ===

submissionyearquarter	pwsid	sample_id	sampling_end_date \
0	2026Q1	OH8554612	OH41973 12/31/2007
1	2026Q1	HI0000243	HI296 12/31/2007
2	2026Q1	OK8005507	OK13167 12/31/2009
3	2026Q1	OK2007062	OK13190 12/31/2009
4	2026Q1	OH7834412	OH17604 12/31/1999

sampling_start_date	reconciliation_id	sample_first_reported_date \
0	07/01/2007	nan 11/13/2007
1	01/01/2005	nan 11/14/2007
2	01/01/2007	nan 11/15/2007
3	01/01/2007	nan 11/15/2007
4	07/01/1999	nan 11/13/2007

sample_last_reported_date	sar_id	contaminant_code	result_sign_code \
0	11/19/2009	13994336	PB90 nan
1	08/18/2008	13996062	PB90 nan
2	02/17/2010	13999114	PB90 nan
3	02/16/2012	13999119	PB90 nan
4	07/13/2009	13992537	PB90 nan

sample_measure	unit_of_measure	sar_first_reported_date \
0	0	mg/L 11/13/2007
1	.005	mg/L 11/14/2007
2	0	mg/L 11/15/2007
3	0	mg/L 11/15/2007
4	0	mg/L 11/13/2007

	sar_last_reported_date
0	11/19/2009
1	08/18/2008
2	02/17/2010
3	02/16/2012
4	07/13/2009

=== Athena preview: facilities ===

	submissionyearquarter	pwsid	facility_id	facility_name	\
0	2026Q1	NC0187122	19835	SPRING	
1	2026Q1	NJ0504349	32880	TREATMENT PLANT	
2	2026Q1	NJ0504350	32878	TREATMENT PLANT	
3	2026Q1	NJ0506315	32872	TREATMENT PLANT	
4	2026Q1	NJ0506419	32879	TREATMENT PLANT	

	state_facility_id	facility_activity_code	facility_deactivation_date	\
0	nan	I	09/01/1993	
1	TP001001	A	nan	
2	TP001001	A	nan	
3	TP001001	A	nan	
4	TP001001	A	nan	

	facility_type_code	submission_status_code	is_source_ind	water_type_code	\
0	WL		Y	Y	GW
1	TP		Y	N	nan
2	TP		Y	N	nan
3	TP		Y	N	nan
4	TP		Y	N	nan

	availability_code	seller_treatment_code	seller_pwsid	seller_pws_name	\
0	P		nan	nan	nan
1	nan		nan	nan	nan
2	nan		nan	nan	nan
3	nan		nan	nan	nan
4	nan		nan	nan	nan

	filtration_status_code	is_source_treated_ind	first_reported_date	\
0	nan	N	nan	
1	nan	nan	04/18/2006	
2	nan	nan	04/18/2006	
3	nan	nan	04/18/2006	
4	nan	nan	04/18/2006	

	last_reported_date
0	12/22/2001
1	03/27/2026
2	03/27/2026
3	09/19/2024
4	12/03/2008

=== Athena preview: service_areas ===

	submissionyearquarter	pwsid	service_area_type_code	\
0	2026Q1	010106001	RA	
1	2026Q1	010109001	HM	
2	2026Q1	010109005	RA	

3	2026Q1	010307001	RA
4	2026Q1	010502002	ON

	is_primary_service_area_code	first_reported_date	last_reported_date
0	Y	11/30/2006	03/25/2026
1	Y	11/30/2006	05/10/2007
2	Y	05/26/2010	03/25/2026
3	Y	11/30/2006	03/25/2026
4	Y	05/26/2010	03/25/2026

=== Athena preview: geographic_areas ===

	submissionyearquarter	pwsid	geo_id	area_type_code	tribal_code \
0	2026Q1	NH0867060	14949232	CN	nan
1	2026Q1	NH0196230	14949233	CT	nan
2	2026Q1	MI2021369	14949234	CN	nan
3	2026Q1	MI2006109	14949235	CN	nan
4	2026Q1	MD1031310	14949236	CN	nan

	state_served	ansi_entity_code	zip_code_served	city_served	county_served \
0	nan	003	nan	nan	Carroll
1	NH	nan	nan	BEDFORD	nan
2	nan	137	nan	nan	Otsego
3	nan	017	nan	nan	Bay
4	nan	005	nan	nan	Baltimore

	last_reported_date
0	06/30/2016
1	06/30/2016
2	06/05/2025
3	06/05/2025
4	05/22/2023

=== Athena preview: ref_codes ===

	value_type	value_code	value_description
0	PRIMACY_AGENCY_CODE	TX	Texas
1	PRIMACY_AGENCY_CODE	UT	Utah
2	PRIMACY_AGENCY_CODE	VA	Virginia
3	PRIMACY_AGENCY_CODE	VI	US Virgin Islands
4	CONTAMINANT_CODE	2071	PARA-PARA DDD

=== Athena preview: ansi_areas ===

	ansi_state_code	ansi_entity_code	ansi_name	state_code
0	12	097	Osceola	FL
1	12	099	Palm Beach	FL
2	12	101	Pasco	FL
3	12	103	Pinellas	FL
4	12	105	Polk	FL

=== Athena preview: pn_assoc ===

	submissionyearquarter	pwsid	pn_violation_id	related_violation_id \
0	2026Q1	010106001	010600175750092018	010600102295072018
1	2026Q1	020000005	0000005757500122020	00000053A8000122020
2	2026Q1	020000005	000000575750072020	00000053A800072020
3	2026Q1	010307001	030700175750072006	030700121310072006
4	2026Q1	010502002	050200275750062009	050200222310062009

	compl_per_begin_date	compl_per_end_date	non_compl_per_begin_date	\
0	07/01/2018	09/30/2018	07/01/2018	
1	12/01/2020	12/31/2020	12/01/2020	
2	07/01/2020	07/31/2020	07/01/2020	
3	07/01/2006	07/31/2006	07/01/2006	
4	06/01/2009	06/30/2009	06/01/2009	

	non_compl_per_end_date	violation_code	contaminant_code	first_reported_date	\
0	09/30/2018	02	2950	03/25/2026	
1	06/19/2018	3A	8000	10/07/2024	
2	06/19/2018	3A	8000	10/07/2024	
3	07/12/2006	21	3100	12/08/2022	
4	06/30/2009	22	3100	12/08/2022	

	last_reported_date
0	nan
1	03/27/2026
2	03/27/2026
3	03/25/2026
4	03/25/2026

=== Athena preview: site_visits ===

	submissionyearquarter	pwsid	visit_id	visit_date	agency_type_code	\
0	2026Q1	IL0995336	2	11/16/1993	ST	
1	2026Q1	IL0995336	3	06/26/1990	ST	
2	2026Q1	IL3021824	#7	11/10/1981	nan	
3	2026Q1	IL3021824	#6	07/29/1983	nan	
4	2026Q1	IL1110400	10	09/29/1997	ST	

	visit_reason_code	management_ops_eval_code	source_water_eval_code	\
0	SNSV	X	X	
1	SNSV	X	X	
2	SNSV	nan	nan	
3	SNSV	nan	nan	
4	SNSV	X	X	

	security_eval_code	pumps_eval_code	other_eval_code	compliance_eval_code	\
0	nan	X	X	X	
1	nan	X	X	X	
2	nan	nan	nan	nan	
3	nan	nan	nan	nan	
4	nan	X	X	X	

	data_verification_eval_code	treatment_eval_code	\
0	X	X	
1	X	X	
2	nan	nan	
3	nan	nan	
4	X	X	

	finished_water_stor_eval_code	distribution_eval_code	financial_eval_code	\
0	X	X	nan	
1	X	X	nan	
2	nan	nan	nan	
3	nan	nan	nan	
4	X	X	nan	

	visit_comments	first_reported_date	\
0	nan	12/09/2005	
1	nan	12/09/2005	
2	nan	07/22/1995	
3	nan	07/22/1995	
4	IDPH: Y FMHA: N ATTACHMENT B: N	12/09/2005	

	last_reported_date
0	12/07/2006
1	12/07/2006
2	nan
3	nan
4	12/07/2006

Joinability Checks for Core SDWA Relational Keys

This cell defines a reusable helper function that measures how well two SDWA tables can be joined on a shared key (typically `pwsid`). For each table pair, it reports both the total number of rows in the left table and how many of those successfully match a row in the right table. The cell then runs joinability checks between PWS and three critical datasets—Violations, LCR Samples, and Service Areas—to validate relational integrity before constructing the PWS master feature table.

```
In [13]: # -----
# 14. Joinability checks for core keys
# -----

def check_key_overlap(table1, key1, table2, key2):
    q = f"""
    SELECT
        COUNT(*) AS total,
        COUNT(CASE WHEN t2.{key2} IS NOT NULL THEN 1 END) AS matched
    FROM {database}.{table1} t1
    LEFT JOIN {database}.{table2} t2
        ON t1.{key1} = t2.{key2}
    """
    return wr.athena.read_sql_query(q, database, workgroup=athena_workgroup, s3_out

print("PWS <--> Violations")
display(check_key_overlap("pws", "pwsid", "violations", "pwsid"))

print("PWS <--> LCR")
display(check_key_overlap("pws", "pwsid", "lcr", "pwsid"))

print("PWS <--> Service Areas")
display(check_key_overlap("pws", "pwsid", "service_areas", "pwsid"))
```

	total	matched
0	15473495	15298031

PWS <--> LCR

	total	matched
0	1268571	924498

PWS <--> Service Areas

	total	matched
0	477670	422099

Aggregate Violations by PWSID Using Athena

This cell computes violation-level summary features for each Public Water System (PWSID). It queries Athena to produce two key aggregates: the total number of violations and the number of distinct violation IDs associated with each system. These features are later joined into the PWS master table for modeling and analysis.

```
In [14]: # -----  
# Violations aggregated by PWSID  
# -----  
viol_agg = wr.athena.read_sql_query(  
    f"""  
    SELECT  
        pwsid,  
        COUNT(*) AS violation_count,  
        COUNT(DISTINCT violation_id) AS distinct_violations  
    FROM {database}.violations  
    GROUP BY pwsid  
    """,  
    database,  
    workgroup=athena_workgroup,  
    s3_output=athena_output  
)
```

LCR Sampling Aggregated by PWSID

This cell computes Lead and Copper Rule (LCR) sampling statistics for each Public Water System (PWSID). It queries Athena to produce two key features: the total number of LCR samples and the average measured sample value (after cleaning placeholder values like `"."`, empty strings, and `"nan"`). These aggregates are later joined into the unified PWS master feature table.

```
In [15]: # -----  
# LCR sampling aggregated by PWSID  
# -----  
lcr_agg = wr.athena.read_sql_query(  
    f"""  
    SELECT
```

```

        pwsid,
        COUNT(*) AS lcr_samples,
        AVG(CAST(NULLIF(sample_measure, '.') AS DOUBLE)) AS avg_lcr_result
    FROM {database}.lcr
    WHERE sample_measure IS NOT NULL
        AND sample_measure NOT IN ('', 'nan')
    GROUP BY pwsid
    """
    database,
    workgroup=athena_workgroup,
    s3_output=athena_output
)

```

Build the Unified PWS-Level Master Feature Table

This cell constructs the full PWS-level master dataset by joining Public Water System metadata with four aggregated feature groups: violations, LCR sampling, service area characteristics, and geographic attributes. Each aggregation is computed in a CTE for clarity and then left-joined to the PWS table to ensure every system is retained, even if some feature groups have no matching records. The resulting table serves as the foundation for all downstream modeling, analysis, and validation steps in the SDWA pipeline.

```

In [16]: pws_master = wr.athena.read_sql_query(
    f"""
    WITH viol_agg AS (
        SELECT
            pwsid,
            COUNT(*) AS violation_count,
            COUNT(*) FILTER (WHERE enforcement_action_type_code IS NOT NULL) AS enf
        FROM {database}.violations
        GROUP BY pwsid
    ),

    lcr_agg AS (
        SELECT
            pwsid,
            COUNT(*) AS lcr_samples,
            AVG(CAST(NULLIF(sample_measure, '.') AS DOUBLE)) AS avg_lcr_result
        FROM {database}.lcr
        WHERE sample_measure IS NOT NULL
            AND sample_measure NOT IN ('', 'nan')
        GROUP BY pwsid
    ),

    service_agg AS (
        SELECT
            pwsid,
            COUNT(*) AS service_area_count,
            MAX(service_area_type_code) AS primary_service_area_type
        FROM {database}.service_areas
        GROUP BY pwsid
    ),
    """
)

```

```

geo_agg AS (
    SELECT
        pwsid,
        MAX(state_served) AS state,
        MAX(county_served) AS county,
        MAX(city_served) AS city,
        MAX(area_type_code) AS area_type
    FROM {database}.geographic_areas
    GROUP BY pwsid
)

SELECT
    pws.pwsid,
    pws.pws_name,
    pws.pws_type_code,
    pws.primacy_agency_code,
    pws.epa_region,
    pws.state_code,
    pws.source_water_protection_code,
    pws.seasonal_startup_system,

    -- Aggregated features
    v.violation_count,
    v.enforcement_actions,
    l.lcr_samples,
    l.avg_lcr_result,
    s.service_area_count,
    s.primary_service_area_type,
    g.state,
    g.county,
    g.city,
    g.area_type

FROM {database}.pws pws
LEFT JOIN viol_agg v ON pws.pwsid = v.pwsid
LEFT JOIN lcr_agg l ON pws.pwsid = l.pwsid
LEFT JOIN service_agg s ON pws.pwsid = s.pwsid
LEFT JOIN geo_agg g ON pws.pwsid = g.pwsid
""",
database,
workgroup=athena_workgroup,
s3_output=athena_output
)

```

Inspect the PWS Master Table Shape and Descriptive Statistics

This cell prints the overall dimensions of the `pws_master` DataFrame and generates a full descriptive summary (including categorical columns) to verify that the unified feature table was constructed correctly. This provides an early sanity check on row counts, missingness patterns, and value distributions before moving into modeling or further feature engineering.

```
In [17]: print("pws_master shape:", pws_master.shape)
pws_master.describe(include="all").transpose()
```

pws_master shape: (433698, 18)

Out[17]:

	count	unique	top	freq	mean	std
pwsid	433698	433698	WI4710292	1	NaN	NaN
pws_name	433698	391511	nan	896	NaN	NaN
pws_type_code	433698	4	TNCWS	287188	NaN	NaN
primacy_agency_code	433698	66	MI	27262	NaN	NaN
epa_region	433698	10	05	127302	NaN	NaN
state_code	433698	65	MI	26703	NaN	NaN
source_water_protection_code	433698	3	nan	314784	NaN	NaN
seasonal_startup_system	433698	6	nan	416892	NaN	NaN
violation_count	258234.0	<NA>	<NA>	<NA>	59.240964	178.817539
enforcement_actions	258234.0	<NA>	<NA>	<NA>	59.240964	178.817539
lcr_samples	89625.0	<NA>	<NA>	<NA>	10.31518	7.880947
avg_lcr_result	89625.0	NaN	NaN	NaN	1.340308	157.829849
service_area_count	378127.0	<NA>	<NA>	<NA>	1.116289	0.465319
primary_service_area_type	378127	28	RS	62436	NaN	NaN
state	413546	31	nan	411671	NaN	NaN
county	413546	1822	nan	160691	NaN	NaN
city	413546	1024	nan	411671	NaN	NaN
area_type	413546	5	CN	256223	NaN	NaN



Clean and Normalize the PWS Master Table (Mixed-Type Safe)

This cell finalizes the construction of the `pws_master` feature table by performing safe, dtype-aware cleanup. It removes duplicate rows, converts all numeric-like columns to proper numeric types using `pd.to_numeric` with `errors="coerce"` (ensuring invalid values become `NaN` instead of raising errors), and fills missing values only in object/string columns. This preserves numeric integrity for modeling while ensuring categorical fields contain no nulls.

```
In [18]: # -----
# Clean up PWS master table (safe for mixed dtypes)
# -----

# Drop duplicates
pws_master = pws_master.drop_duplicates()

# Identify numeric columns
numeric_cols = [
    "violation_count",
    "enforcement_actions",
    "lcr_samples",
    "avg_lcr_result",
    "service_area_count"
]

# Convert numeric-like strings to numeric first
for col in numeric_cols:
    if col in pws_master.columns:
        pws_master[col] = pd.to_numeric(pws_master[col], errors="coerce")

# Fill NA only in object/string columns
string_cols = pws_master.select_dtypes(include=["object"]).columns
pws_master[string_cols] = pws_master[string_cols].fillna("")
```

Train/Val/Test/Prod Split (40/10/10/40)

This cell shuffles the fully-cleaned `pws_master` feature table and partitions it into four disjoint subsets:

- **40%** training
- **10%** validation
- **10%** test
- **40%** production holdout (never used during model development)

The split is deterministic via `random_state=42`, ensuring reproducibility across runs and environments. These partitions feed directly into the modeling and evaluation pipeline.

```
In [19]: # -----
# 40% train, 10% val, 10% test, 40% prod
# -----

pws_master = pws_master.sample(frac=1, random_state=42)

n = len(pws_master)
train_end = int(0.4 * n)
val_end   = int(0.5 * n)
test_end  = int(0.6 * n)

train = pws_master.iloc[:train_end]
val    = pws_master.iloc[train_end:val_end]
test   = pws_master.iloc[val_end:test_end]
```

```
prod = pws_master.iloc[test_end:]

print(len(train), len(val), len(test), len(prod))
```

173479 43370 43369 173480

Write PWS Master Splits to S3 as Parquet

This cell saves each of the four PWS-level splits—train, validation, test, and production—to S3 as Parquet datasets. Each split is written to its own prefix under `pws_master/splits/`, using `mode="overwrite"` to ensure idempotent, re-runnable behavior. These Parquet outputs become the canonical, versionable artifacts for downstream modeling, evaluation, and deployment workflows.

```
In [20]: # -----
# Write PWS master splits to S3 as Parquet
# -----
def write_split(name, df):
    path = f"s3://{bucket}/pws_master/splits/{name}/"
    print(f"Writing {name} → {path}")
    wr.s3.to_parquet(
        df=df,
        path=path,
        dataset=True,
        mode="overwrite"
    )

write_split("train", train)
write_split("val", val)
write_split("test", test)
write_split("prod", prod)
```

Writing train → s3://aai540-group5-public-866792937762-us-east-1-an/pws_master/splits/train/

Writing val → s3://aai540-group5-public-866792937762-us-east-1-an/pws_master/splits/val/

Writing test → s3://aai540-group5-public-866792937762-us-east-1-an/pws_master/splits/test/

Writing prod → s3://aai540-group5-public-866792937762-us-east-1-an/pws_master/splits/prod/

Register PWS Master Split Tables in Athena

This cell finalizes the split-registration workflow by looping over the four PWS master subsets—**train**, **val**, **test**, and **prod**—and registering each one as an Athena table.

Each split is written to its own S3 prefix under `pws_master/splits/`, and `safe_create_parquet_table` ensures:

- schema inference from a sample Parquet file
- all columns coerced to **STRING** for Glue/Athena stability
- idempotent overwrite behavior

These Athena tables become the queryable interfaces for validation, feature inspection, and downstream ML workflows.

```
In [21]: # -----  
# Register PWS master split tables in Athena  
# -----  
for split_name in ["train", "val", "test", "prod"]:  
    path = f"s3://{bucket}/pws_master/splits/{split_name}/"  
    table_name = f"pws_master_{split_name}"  
    print(f"Registering Athena table: {table_name}")  
    safe_create_parquet_table(database, table_name, path)
```

Registering Athena table: pws_master_train

[START] Registering Athena table 'pws_master_train'

S3 Path: s3://aai540-group5-public-866792937762-us-east-1-an/pws_master/splits/train/

- Using sample file: s3://aai540-group5-public-866792937762-us-east-1-an/pws_master/splits/train/5012dfa285f64dddbad56ede35d9a754.snappy.parquet

- Columns detected: 18 (all coerced to STRING)

[DONE] Table 'pws_master_train' registered successfully in 2.83s

Registering Athena table: pws_master_val

[START] Registering Athena table 'pws_master_val'

S3 Path: s3://aai540-group5-public-866792937762-us-east-1-an/pws_master/splits/val/

- Using sample file: s3://aai540-group5-public-866792937762-us-east-1-an/pws_master/splits/val/0933a83e0b8c47b58d8cc8ca943ef3f9.snappy.parquet

- Columns detected: 18 (all coerced to STRING)

[DONE] Table 'pws_master_val' registered successfully in 1.68s

Registering Athena table: pws_master_test

[START] Registering Athena table 'pws_master_test'

S3 Path: s3://aai540-group5-public-866792937762-us-east-1-an/pws_master/splits/test/

- Using sample file: s3://aai540-group5-public-866792937762-us-east-1-an/pws_master/splits/test/9b410c60a692471cbea3630d837a013d.snappy.parquet

- Columns detected: 18 (all coerced to STRING)

[DONE] Table 'pws_master_test' registered successfully in 2.39s

Registering Athena table: pws_master_prod

[START] Registering Athena table 'pws_master_prod'

S3 Path: s3://aai540-group5-public-866792937762-us-east-1-an/pws_master/splits/prod/

- Using sample file: s3://aai540-group5-public-866792937762-us-east-1-an/pws_master/splits/prod/f3a5d1c67d764184bf82aa55f445bf59.snappy.parquet

- Columns detected: 18 (all coerced to STRING)

[DONE] Table 'pws_master_prod' registered successfully in 1.91s

Verify Row Counts for All PWS Master Splits

This final validation step confirms that each PWS master split—**train**, **val**, **test**, and **prod**—was successfully written to S3 *and* correctly registered as an Athena table. For each split, the cell issues a simple `COUNT(*)` query against the corresponding Athena table and prints the resulting row count.

Matching counts between the in-memory DataFrame partitions and the Athena tables ensures the split pipeline is lossless, deterministic, and ready for downstream ML workflows.

```
In [22]: # -----  
# Verify counts match for PWS master splits  
# -----  
for split in ["train", "val", "test", "prod"]:  
    q = f"SELECT COUNT(*) AS rows FROM {database}.pws_master_{split}"  
    result = wr.athena.read_sql_query(  
        q,  
        database,  
        workgroup=athena_workgroup,  
        s3_output=athena_output  
    )  
    print(split, result["rows"][0])
```

```
train 173479  
val 43370  
test 43369  
prod 173480
```